



Введение в FESTIM v2.x

Владимир Кулагин

Установка окружения

00 / Подготовка

01 Скачать материалы:

```
$ git clone https://github.com/KulaginVladimir/FESTIM-Introduction.git
```

02 Установить окружение:

```
$ conda env create -f environment.yml
```

03 Активировать окружение:

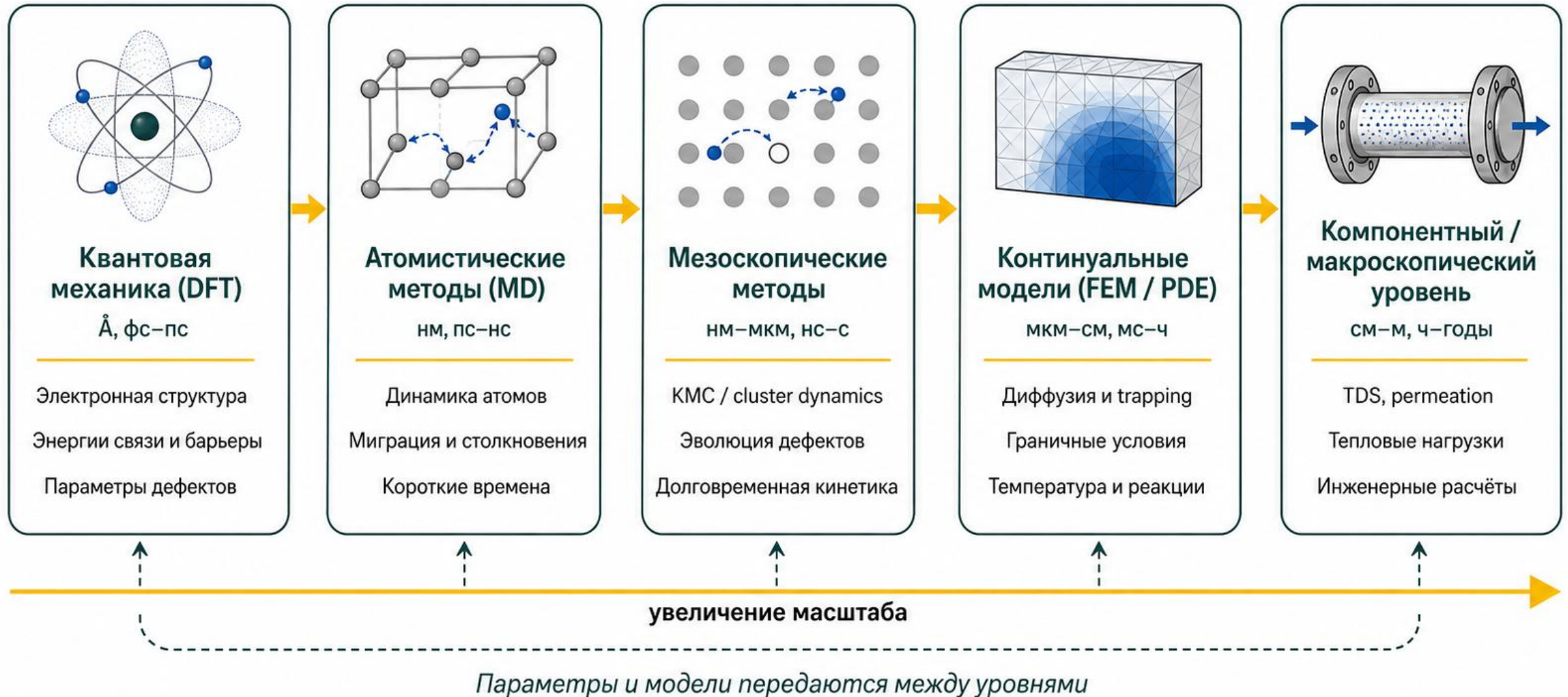
```
$ conda activate festim-introduction
```

04 Проверить:

```
$ python -c "import festim as F; print(F.__version__)"
```

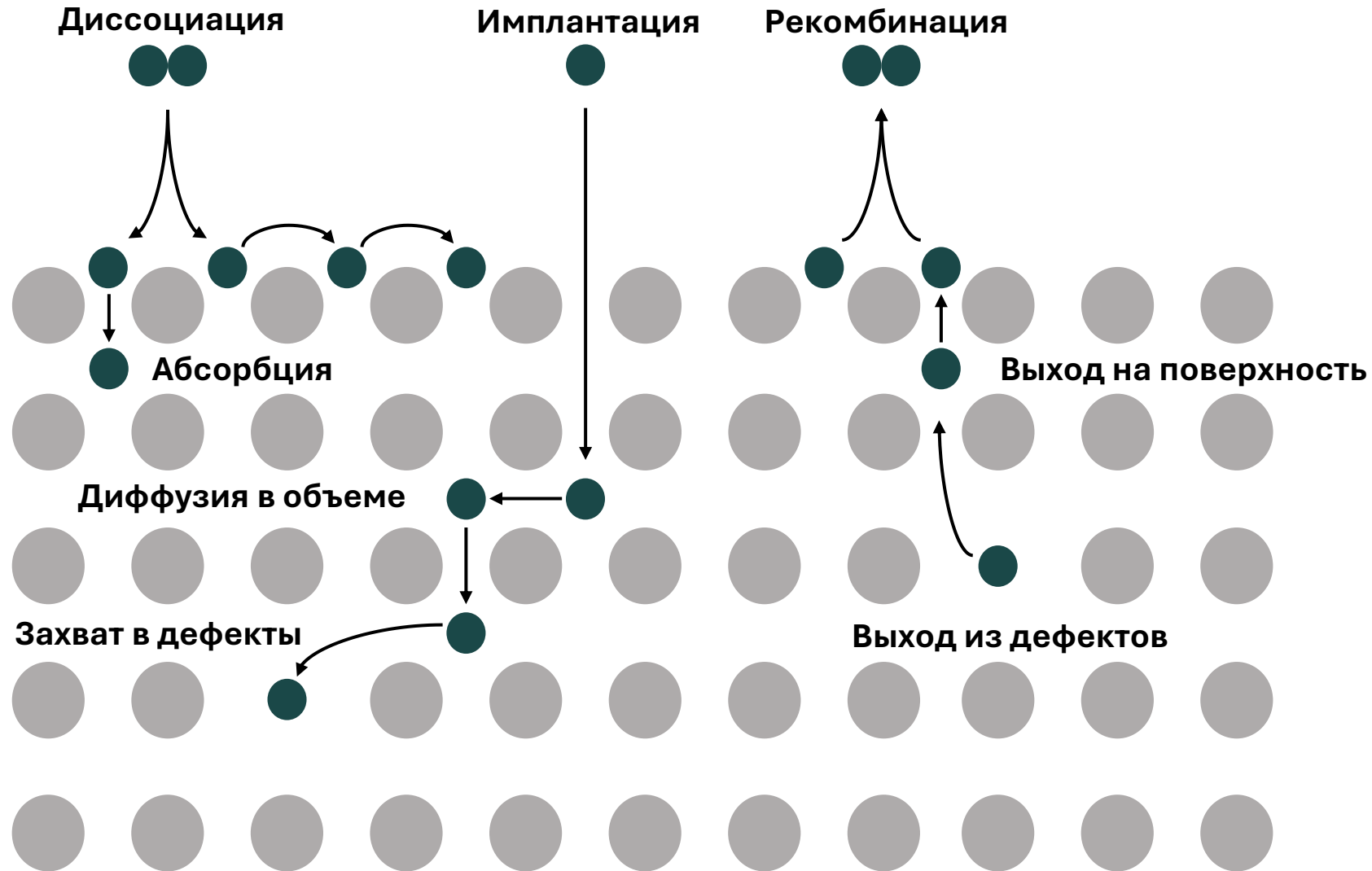
Подходы к моделированию

01 / Введение



Перенос атомов Н в материалах

01 / Введение



Диффузия подвижных атомов

01 / Введение

Поток атомов:

$$\mathbf{J} = -D\nabla c_m$$

Баланс атомов:

$$\frac{\partial c_m}{\partial t} = -\nabla \cdot \mathbf{J} + S$$

Имплантация:
 $S = \Gamma f(x)$

$\mathbf{J}$ — плотность потока подвижных атомов

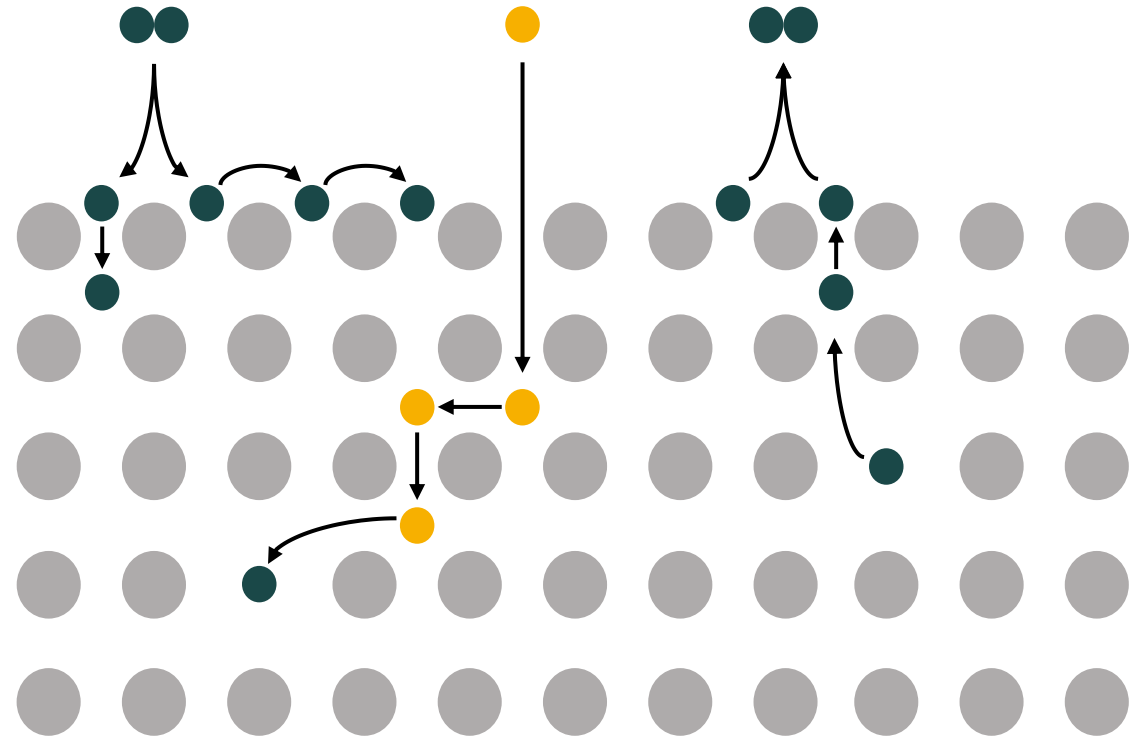
c_m — концентрация подвижных атомов

D — коэффициент диффузии

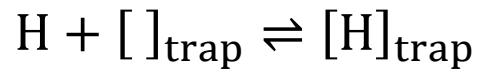
S — объемный источник подвижных атомов

Γ — плотность потока внедренных атомов

$f(x)$ — профиль внедренных атомов



Захват и выход из «ловушек»:



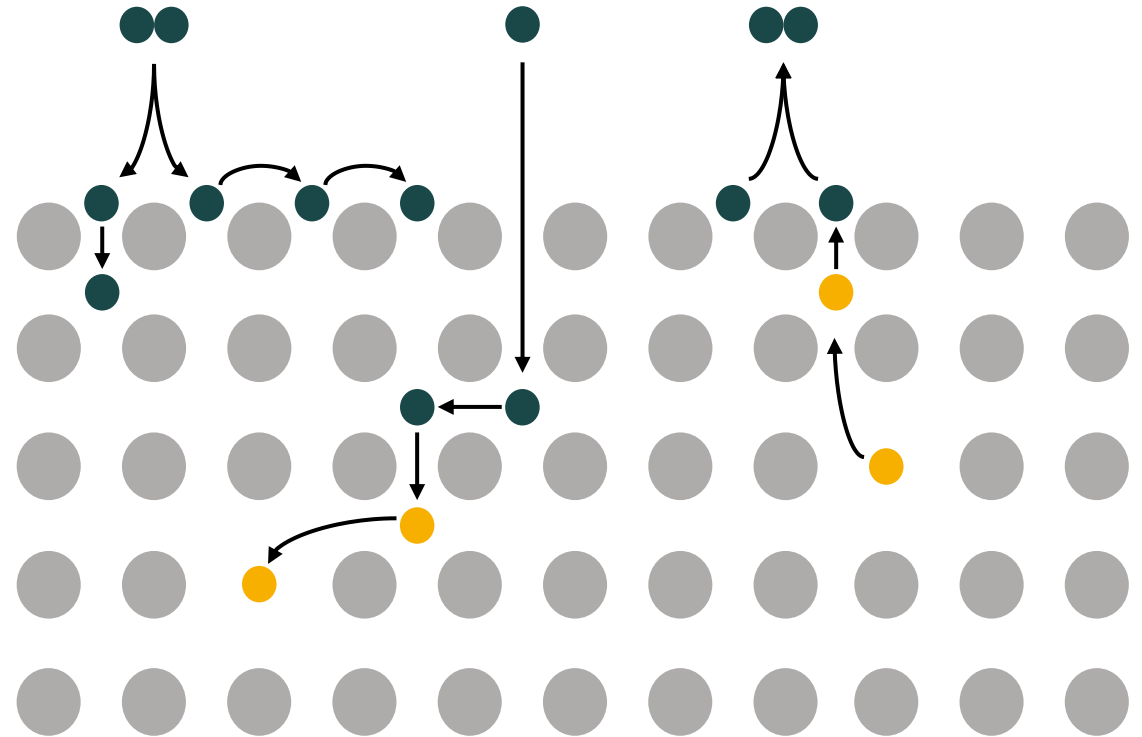
$$\frac{\partial c_t}{\partial t} = kc_m(n_t - c_t) - pc_t$$

n_t — концентрация «ловушек»

c_t — захваченные атомы

k — константа скорости захвата в «ловушки»

p — константа скорости выхода из «ловушек»



Баланс потоков:
$$-\mathbf{J} \cdot \mathbf{n} = \Gamma_{\text{diss}} - \Gamma_{\text{rec}}$$

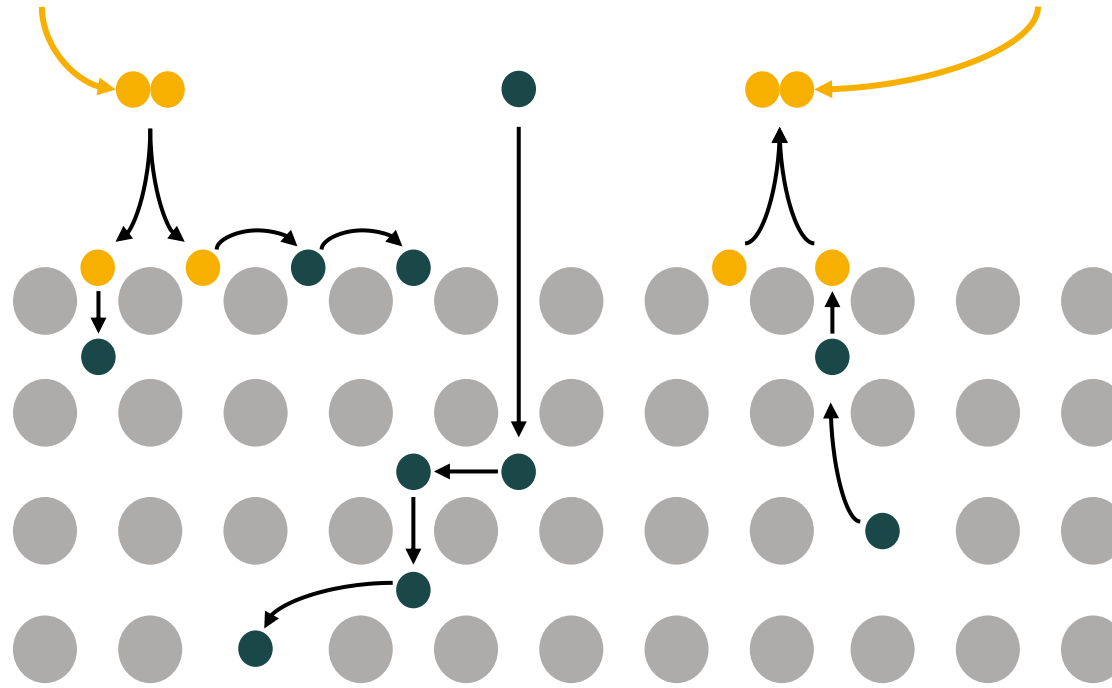
Быстрая десорбция:
$$c_m = 0$$

Диссоциация молекул:

$$\Gamma_{\text{diss}} = K_d P$$

Рекомбинация атомов:

$$\Gamma_{\text{rec}} = K_r c_m^2$$



Перенос тепла

01 / Введение

Большинство процессов протекает быстрее с ростом температуры материала

$$D = D_0 \exp\left(-\frac{E_d}{kT}\right)$$


$$C\rho \frac{\partial T}{\partial t} = \nabla \cdot (\kappa \nabla T) + \sum_i Q_i$$

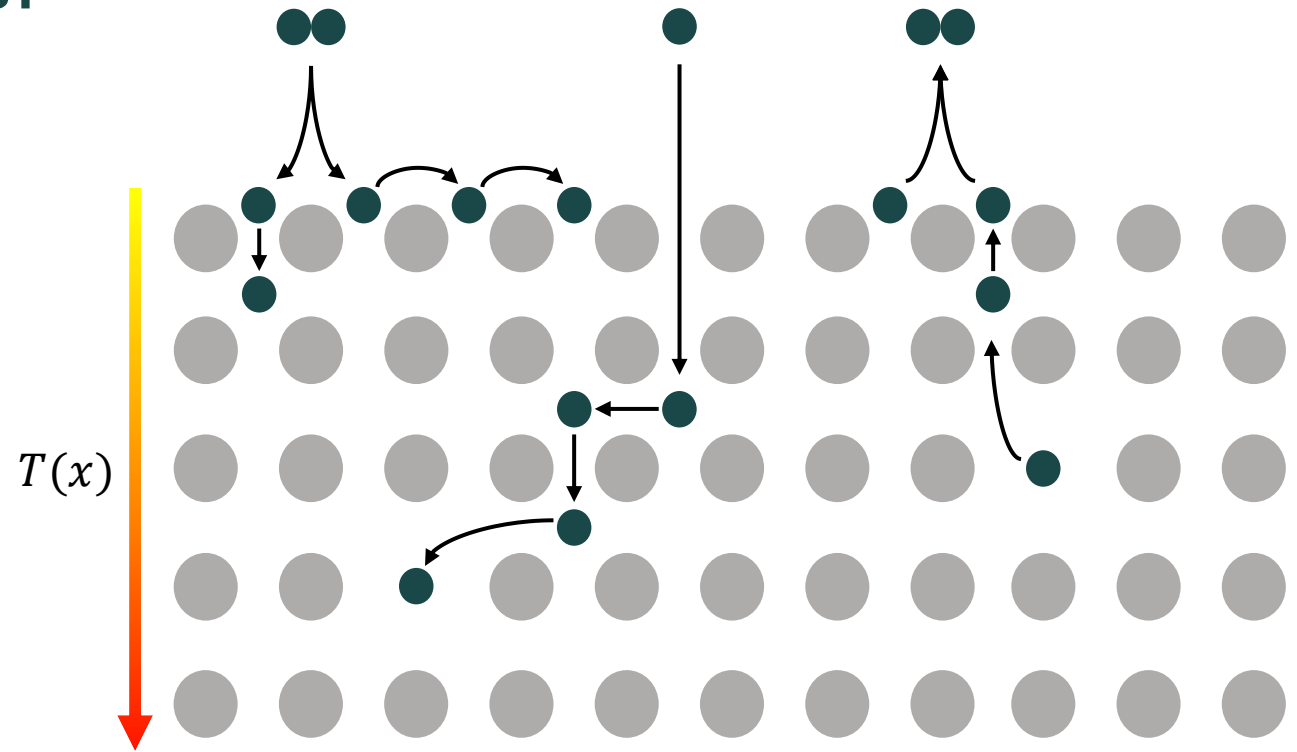
C — теплоемкость

ρ — плотность

κ — теплопроводность

T — температура

Q — объемный источник мощности



FESTIM и сообщество



FESTIM (Finite Element Simulation of Tritium in Materials) – **свободно** распространяемая библиотека для моделирования переноса изотопов водорода в материалах



FESTIM реализован как библиотека **Python**, что позволяет гибко задавать физические модели непосредственно в коде



Решение уравнений в частных производных проводится на основе **метода конечных элементов** при помощи открытой библиотеки **FEniCSx**

FESTIM

Перенос изотопов водорода

$$\frac{\partial c_m}{\partial t} = -\nabla \cdot \mathbf{J} - \sum_i \frac{\partial c_{t,i}}{\partial t} + \sum_j S_j$$

$$\frac{\partial c_{t,i}}{\partial t} = R_{\text{trap},i} - R_{\text{detrap},i}$$

Перенос тепла

$$c_p \frac{\partial T}{\partial t} = \nabla \cdot (\kappa \nabla T) + \sum_i Q_i$$

Почему FESTIM?

02 / FESTIM



Открытый и расширяемый код
[festim-dev/FESTIM](https://github.com/festim-dev/FESTIM)



Верифицирован и валидирован:
[festim-dev/V-V-report](https://github.com/festim-dev/V-V-report)



Используется в исследованиях

- 30+ публикаций
- ≈ 250 цитирований

Рабочая 1D-модель
≈ 30 строк кода

```
import festim as F
import numpy as np

my_model = F.HydrogenTransportProblem()

my_model.mesh = F.Mesh1D(vertices=np.linspace(0, 1, num=1001))

mat = F.Material(D_0=1, E_D=0.0)

volume_subdomain = F.VolumeSubdomain1D(id=1, borders=[0, 1],
material=mat)
boundary_left = F.SurfaceSubdomain1D(id=1, x=0)
boundary_right = F.SurfaceSubdomain1D(id=2, x=1)
my_model.subdomains = [volume_subdomain, boundary_left, boundary_right]

H = F.Species("H")
my_model.species = [H]

my_model.temperature = 300

my_model.boundary_conditions = [
    F.FixedConcentrationBC(subdomain=boundary_left, value=1,
species=H),
    F.FixedConcentrationBC(subdomain=boundary_right, value=0,
species=H),
]

my_model.settings = F.Settings(atol=1e-10, rtol=1e-10, final_time=2)

my_model.settings.stepsize = F.Stepsize(0.05)

my_model.initialise()

my_model.run()
```



Документация

API, теория, инструкции для пользователей и разработчиков



FESTIM Workshop

Примеры и практические задания



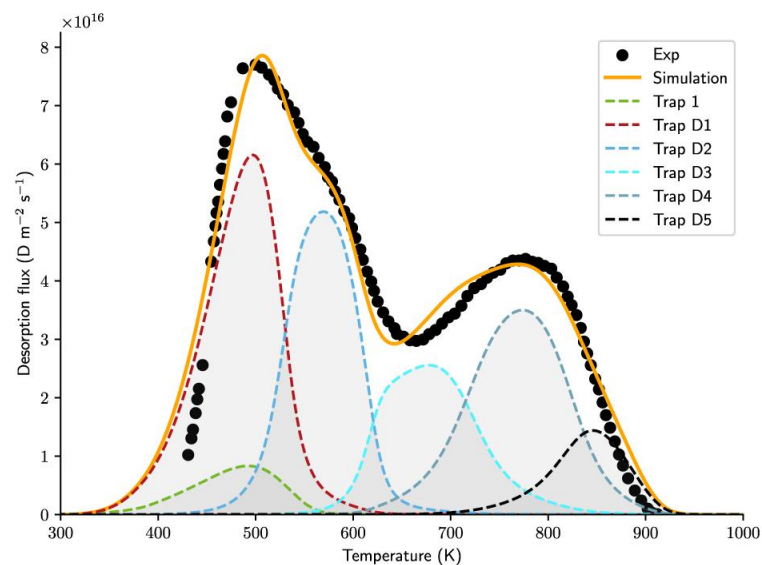
Вопросы пользователей и поиск решений



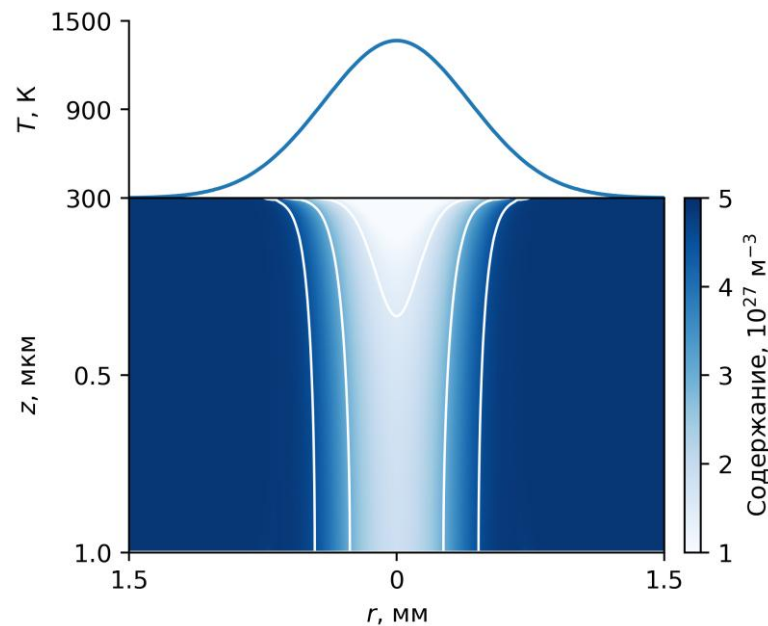
Общение с пользователями и разработчиками

Все ресурсы доступны через
GitHub-репозиторий FESTIM

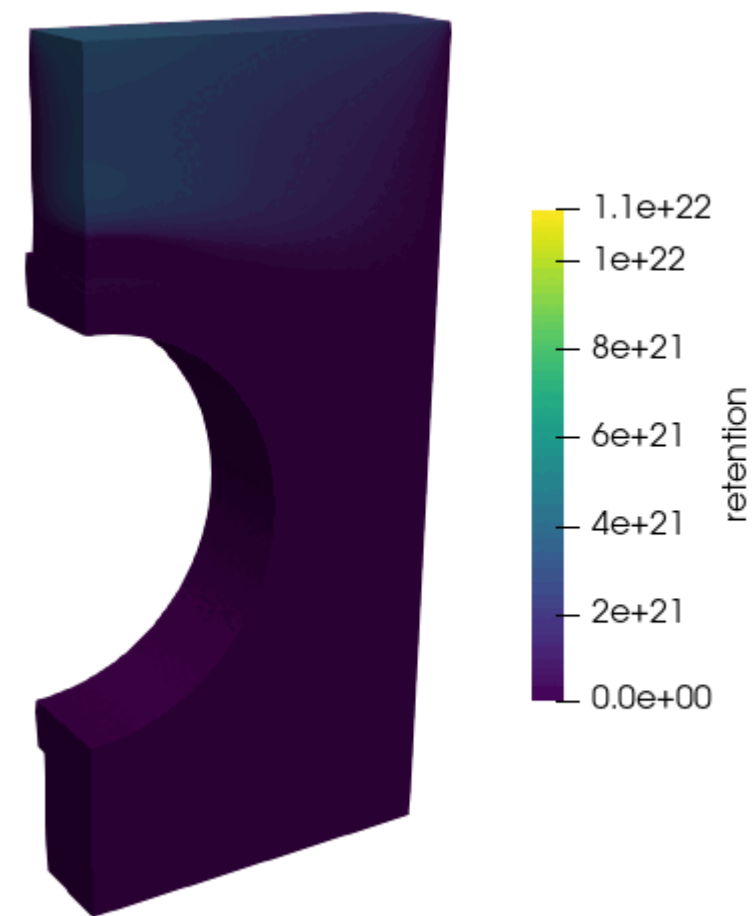
ТДС при медленном нагреве



Десорбция при импульсном нагреве



Накопление в моноблоке



Перенос в WCLL бридере

Time: 1.0e+00 s



Тutorial

`./notebooks/fundamentals`

«Рецепт» модели в FESTIM

03 / Базовые элементы

Входные параметры

- Сетка
- Материалы
- Виды и реакции
- Граничные и начальные условия
- Температура
- Экспорт
- Параметры расчета



FESTIM

```
# Инициализация и  
# запуск расчета  
model.initialise()  
model.run()
```



Результаты

- Поля концентраций
- Поле температуры
- Потоки через поверхности
- Интегральные величины

```
import festim as F
import numpy as np

vertices = np.concatenate(
    [
        np.linspace(0, 1e-7, num=200),
        np.linspace(1e-7, 2e-6, num=1000),
        np.linspace(2e-6, 3e-6, num=200),
    ]
)

mesh = F.Mesh1D(vertices=vertices)
```

```
mat = F.Material(  
    D_0=1.11e-6, # м²/с  
    E_D=0.4, # эВ  
    K_S_0 = 1.0,  
    E_K_S = 3.0,  
    solubility_law = "sievert",  
    thermal_conductivity=100, # Вт/(м К)  
    heat_capacity = 500, # Дж/(кг·К)  
    density = 8000, # кг/м³  
)
```

Виды частиц и реакции

03 / Базовые элементы

```
mobile_H = F.Species(
    name="H",
)

trapped_H = F.Species(
    name="H_trapped",
    mobile=False,
)

empty_traps = F.ImplicitSpecies(
    name="empty_traps",
    n=2.0,
    others=[trapped_H],
)

model = F.HydrogenTransportProblem()
model.species = [mobile_H, trapped_H]
```

```
...

model.reactions = [
    F.Reaction(
        reactant=[mobile_H, empty_traps],
        product=[trapped_H],
        k_0=0.01,
        E_k=0.0,
        p_0=0.1,
        E_p=0.0,
        volume=volume_subdomain,
    )
]
```

```
left_BC = F.HenrysBC(  
    subdomain=left,  
    H_0=1.5,  
    E_H=0.2,  
    pressure=1e5,  
    species=H,  
)
```

```
model.boundary_conditions = [left_BC]
```

```
IC = F.InitialConcentration(  
    value=1e18,  
    species=H,  
    volume=volume,  
)  
  
model.initial_conditions = [IC]
```

```
# Заданная температура
```

```
my_problem.temperature = (lambda x, t: 300.0 + 100.0 * x[0] / L + 2.0 * t)
```

```
# Отдельный расчёт теплопереноса
```

```
heat_model = F.HeatTransferProblem()
```

```
...
```

```
heat_model.initialise()
```

```
heat_model.run()
```

```
my_problem.temperature = heat_model.u
```

```
# Связанный нестационарный расчёт
```

```
coupled_model = F.CoupledTransientHeatTransferHydrogenTransport(
```

```
    heat_problem=heat_model,
```

```
    hydrogen_problem=my_problem,
```

```
)
```

```
coupled_model.initialise()
```

```
coupled_model.run()
```

```
profile = F.Profile1DExport(  
    field=H,  
    subdomain=volume,  
)  
average_concentration = F.AverageVolume(  
    field=H,  
    volume=volume,  
    filename="average_concentration.csv",  
)  
  
right_flux = F.SurfaceFlux(  
    field=H,  
    surface=right,  
    filename="right_flux.csv",  
)  
  
model.exports = [profile, average_concentration, right_flux]
```

```
model.settings = F.Settings(  
    atol=1e-10,  
    rtol=1e-8,  
    final_time=100.0,  
    stepsize=F.Stepsize(  
        initial_value=0.1,  
        growth_factor=1.2,  
        cutback_factor=0.5,  
        target_nb_iterations=4,  
        max_stepsize=5.0,  
        milestones=[10.0, 20.0, 50.0],  
    ),  
)
```

Примеры задач

`./notebooks/applications`

01 Нестационарная задача с фиксированными граничными условиями

02 Проницаемость водорода через материал

03 ТДС (Десорбция при известной линейной скорости нагрева материала) после имплантации

04 Решение обратной задачи на основе ТДС-эксперимента

Решение обратной задачи

04 / Примеры

